Extensión IoT del Lab 2 · Guía de conexiones

SINAMICS V20 + ESP32 · consigna AI1 aislada + mando por DI + estado por DO + corriente de fase con ACS712 + app móvil MQTT · Firmware `v20_iot_esp32.ino` v3.1

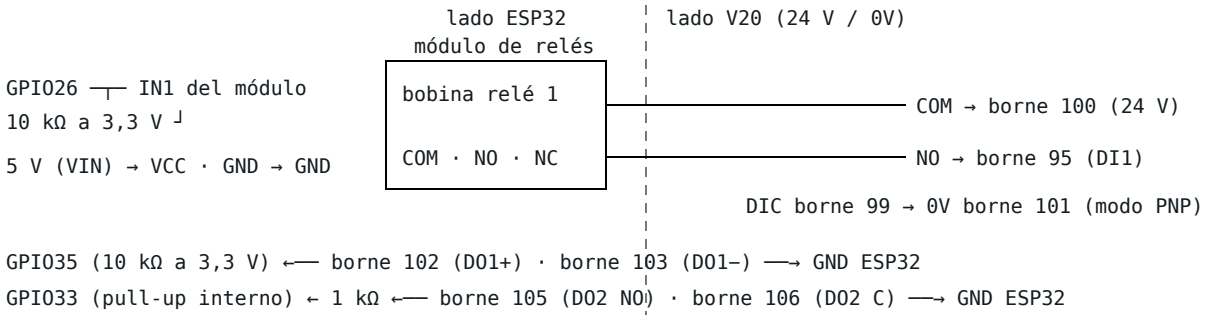
1. Qué se agrega al circuito del Lab 2

Función	Del ESP32	Elemento de aislamiento	Al V20 / módulo
Consigna 0–60 Hz	GPIO25	6N137 + acondicionador (sin cambios)	AI1 borne **89**, 0V borne **92**
Marcha / paro	GPIO26	Relé 1 (contacto seco)	DI1 borne **95**
Sentido (atrás)	GPIO27	Relé 2	DI2 borne **96**
Reconocer falla	GPIO32	Relé 3	DI3 borne **97**
V20 en marcha (entrada)	GPIO35	Transistor DO1 del V20 (aislado 500 V)	DO1 bornes **102** / **103**
V20 en falla (entrada)	GPIO33	Relé DO2 del V20 (aislado 4 kV)	DO2 bornes **105** / **106**
Corriente fase U	GPIO34	ACS712 (efecto Hall, aislado)	Entre borne **83** (V20 U) y borne **1** (motor U1)

Materiales nuevos: módulo de relés **electromecánicos** de 5 V de 2 o 4 canales (los azules tipo SRD-05VDC), 4 × 10 kΩ, 1 × 1 kΩ, 1 × 22 nF (223) y el ACS712 del Lab 1. La versión de **5 A** del ACS712 es mejor que la de 30 A: da 185 mV/A, 2,8 veces más señal.

> **No sirve el módulo de relés de estado sólido del Lab 1** (G3MB-202P y similares): su salida es un triac para corriente alterna y no conmuta bien los 24 V de corriente continua de las entradas del V20. **Tampoco sirve el 6N137 para los DI:** su salida soporta como máximo 7 V y el V20 trabaja con 24 V. El 6N137 sigue solo en la consigna.

2. Esquema de un canal de mando (DI) y de las entradas de estado



Los relés 2 y 3 son iguales al 1: `GPIO27 → IN2`, contacto NO al borne 96, y `GPIO32 → IN3`, contacto NO al borne 97. Los COM de los tres relés van al borne 100. Los contactos NC quedan libres.

3. Tabla de cableado pin a pin

Desde	Hasta	Nota	✓
ESP32 GPIO26 / 27 / 32	IN1 / IN2 / IN3 del módulo de relés	Cada uno con 10 kΩ a 3,3 V: relés abiertos durante el reset	☐
ESP32 VIN (5 V) y GND	VCC y GND del módulo de relés	Si el módulo tiene jumper JD-VCC, déjelo puesto	☐
COM de relé 1, 2 y 3	Borne **100** (24 V del V20)	Un puente entre los tres COM	☐
NO de relé 1 / 2 / 3	Bornes **95** (DI1) / **96** (DI2) / **97** (DI3)		☐

Borne 99 (DI C)	Borne **101** (0V)	Entradas en modo PNP (activas con 24 V)	☐
GPIO35	Borne **102** (DO1+) y 10 kΩ a 3,3 V	DO1 es polarizado: + hacia el ESP32	☐
GND del ESP32	Borne **103** (DO1−)	El transistor de DO1 es libre de potencial	☐
GPIO33	1 kΩ → borne **105** (DO2 NO)	Pull-up interno; el contacto cierra con falla	☐
GND del ESP32	Borne **106** (DO2 común)	Contacto seco del relé del V20	☐
ACS712 VCC / GND	5 V (VIN) / GND del ESP32		☐
ACS712 OUT	10 kΩ → GPIO34; en GPIO34: 20 kΩ ‖ 22 nF a GND	El 22 nF (fc ≈ 1 kHz) quita la portadora PWM del V20	☐
Borne 83 (V20 U)	ACS712 IP+ → IP− → borne **1** (motor U1)	Cable de potencia con terminales; nunca protoboard	☐
Borne 84 / 85 (V20 V / W)	Borne 2 / 3 (motor V1 / W1)		☐
Delta	Puentes 7–12, 8–10, 9–11	U1–W2, V1–U2, W1–V2	☐

Comprobación de DO1: con el V20 en marcha, GPIO35 debe medir menos de 0,5 V respecto a GND del ESP32. Si mide más, el transistor de DO1 no satura con 0,33 mA: ponga un PC817 entre DO1 y el ESP32 (LED con 2,2 kΩ desde el borne 100).

Si el módulo se activa en ALTO (algunos tienen jumper H/L), cambie en el firmware `#define RELE_ACTIVO_BAJO 1` por `0`.

4. Parámetros del V20 para el modo IoT

Parámetro	Valor	Función
`P0700`	`2`	Órdenes por bornes (DI). El botón de marcha del panel queda inactivo.
`P1000`	`2`	Consigna de frecuencia por AI1
`P0701`	`1`	DI1: ON/OFF1 (marcha con P1120, paro con P1121)
`P0702`	`12`	DI2: inversión de sentido
`P0703`	`9`	DI3: reconocimiento de falla
`P0731`	`52.2`	DO1: convertidor en marcha (fábrica: 52.3)
`P0732`	`52.3`	DO2: falla activa (fábrica: 52.7)
`P0756[0] … P0760[0], P2000`	`igual que la guía`	Escalado de AI1; P0759[0] = r0752 con consigna de 60 Hz
`r0722`	`lectura`	Estado de las entradas digitales: sirve para comprobar cada DI

Valores verificados contra las tablas de macros de conexión del manual del V20 (Cn002: P0701 = 1, P0702 = 12, P0703 = 9; Cn001/Cn002: P0731 = 52.2, P0732 = 52.3). Confirme la lista completa en el capítulo de parámetros según el firmware del variador.

5. Pruebas antes de arrancar el motor (en este orden)

#	Prueba	Resultado esperado	✓
1	Breaker OFF. Continuidad entre GND del ESP32 y borne 101 (0V)	Abierto (OL). Si marca continuidad, algo une las tierras	☐
2	Breaker ON, motor sin orden. App → "Conexión al broker" → Conectar	Chips "broker: conectado" y "ESP32: en línea"; estado FALLO con "RETORNO ALIMENTACION"	☐
3	RESET en la app; mirar r0722 en el V20	Estado DETENIDO; durante 0,3 s r0722 muestra DI3 activa	☐
4	Con consigna 0 Hz: MARCHA y PARO desde la app; escuchar el relé 1 y medir entre borne 95 y 101	≈ 24 V con marcha; 0 V con paro. r0722 muestra DI1	☐

5	Consigna 30 Hz y MARCHA	Motor acelera en 5 s (P1120 = 10 s); LED "V20 en marcha" encendido; corriente en la app ≈ r0027	☐
6	Atrás con el motor en marcha	Aviso "SENTIDO RECHAZADO EN MARCHA"	☐
7	PARO, esperar a 0 Hz, Atrás, MARCHA	El motor gira al revés; r0021 negativo	☐
8	EMERGENCIA desde la app	DI1 abierto en menos de 0,1 s; el V20 desacelera con P1121; estado FALLO	☐
9	Comparar la corriente de la app con r0027 a 15, 30, 45 y 60 Hz	Diferencia menor de 10 %; anotar para el informe	☐

Seguridad: El ACS712 queda a la tensión de salida del variador (PWM de hasta ≈ 310 V de pico). Conéctelo con el breaker en OFF, con terminales atornillados y la tarjeta sobre base aislante. No conecte contactores al motor mientras el V20 lo alimenta. La EMERGENCIA de la app es una parada por software (OFF1 con rampa); la parada de seguridad sigue siendo el breaker del módulo.

6. Poner la app en el celular

1. Copie `app/index.html` a la carpeta `docs/` del repositorio de GitHub y active **Settings → Pages** con la rama main y la carpeta `/docs`. Queda en `https://usuario.github.io/repositorio/`. Otra opción: copiar `index.html` al celular y abrirlo con Chrome.
2. En HiveMQ Cloud, cree una credencial para la app (Access Management → nueva credencial con permiso de publicar y suscribir).
3. En la app, abra "Conexión al broker": servidor del clúster, puerto **8884**, ruta **/mqtt**, TLS activado, usuario, contraseña y el mismo prefijo de `credenciales.h` (por ejemplo `jair/v20`).
4. En Chrome: menú ⋮ → "Agregar a pantalla de inicio". Se abre como una app.

7. Qué publica y qué escucha el ESP32

Tópico	Sentido	Contenido
`<prefijo>/cmd`	app → ESP32	START, STOP, EMERGENCY, RESET, FWD, REV, PERFIL, PERFIL_STOP, CALIBRATE
`<prefijo>/consigna`	app → ESP32	Frecuencia en Hz (0–60)
`<prefijo>/config`	app → ESP32	rampa=, in=, fmin=, fmax=, vigilancia=
`<prefijo>/estado`	ESP32 → app	DETENIDO, MARCHA o FALLO (retenido)
`<prefijo>/alarma`	ESP32 → app	Causa de la falla (retenido)
`<prefijo>/aviso`	ESP32 → app	Órdenes rechazadas e información
`<prefijo>/telemetria`	ESP32 → app	JSON cada segundo con 26 variables
`<prefijo>/online`	ESP32 → app	ONLINE / OFFLINE (testamento)

8. Protecciones que ejecuta el ESP32

Condición	Detección	Acción
Reinicio del ESP32	Al arrancar	FALLO "RETORNO_ALIMENTACION": exige RESET
Falla del V20	DO2 cerrado	FALLO "FALLA_V20"; RESET da pulso en DI3
V20 sin energía o sin responder	DI1 activo y DO1 abierto durante 2 s	FALLO "V20_SIN_MARCHA"
Fase abierta / motor desconectado	V20 en marcha > 5 Hz e I < máx(0,35 A; 0,15·In) durante 2 s	FALLO "PERDIDA_CORRIENTE_FASE"
Sobrecorriente	I > 2,5·In durante 0,5 s	FALLO "SOBRECORRIENTE"
Sobrecarga	Imagen térmica I^2t, τ = 300 s, disparo θ = 1,2	Advertencia a 0,9; FALLO "SOBRECARGA_I2T"; memoria térmica

Sensor desconectado	Media de la salida fuera de 2,2–2,8 V	FALLO en marcha; impide arrancar
Pérdida de red	Sin MQTT 10 s con marcha ordenada desde la app	FALLO "PERDIDA_COMUNICACION"
Orden retenida	Órdenes en los 1,5 s posteriores a suscribirse	Se ignoran y se avisa
Inversión en marcha	Orden FWD/REV con el motor girando	Se rechaza: evita frenado regenerativo sin resistencia